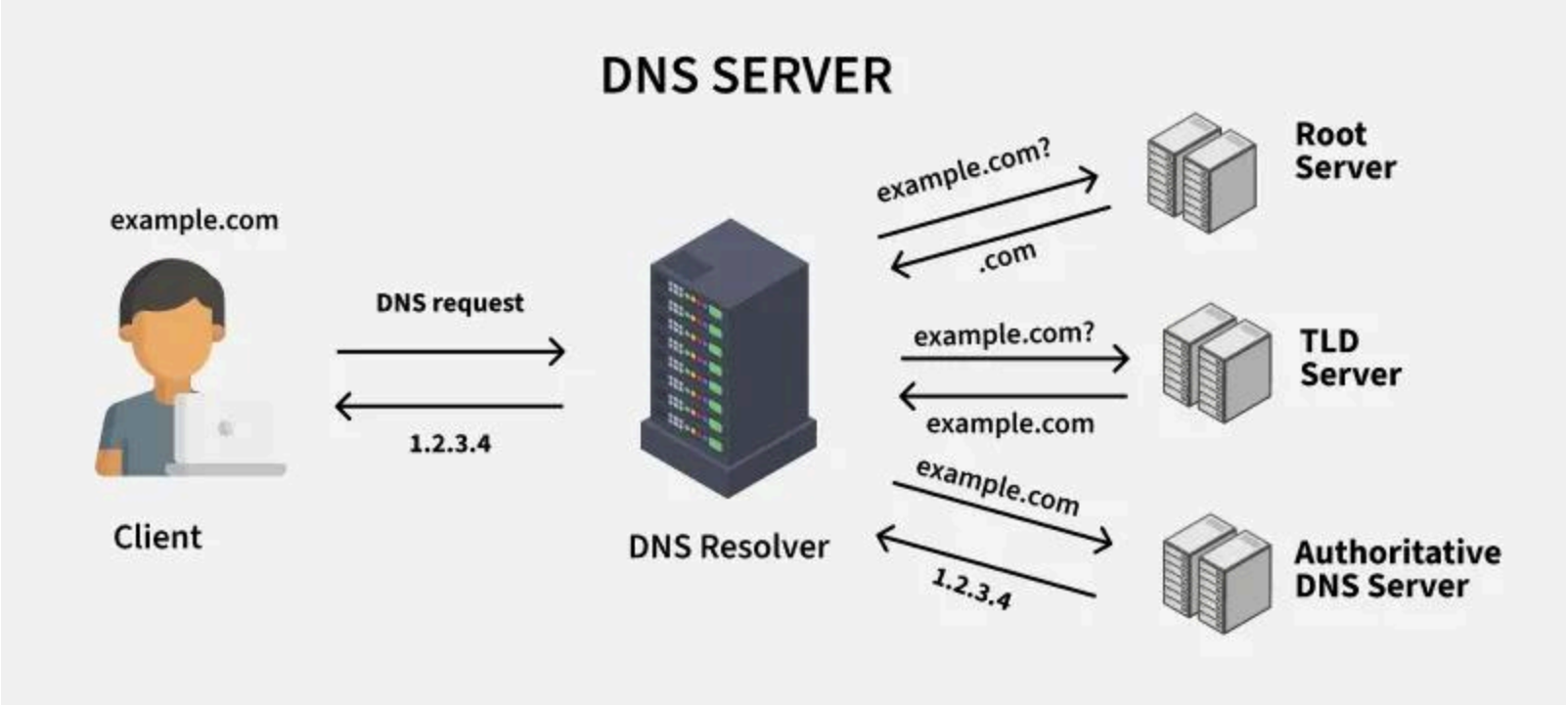
8. DNS – Domain Name System

The Core Idea – Why DNS Exists

- Computers communicate using IP addresses – numbers like 142.250.190.46
- Humans remember names – google.com, not 142.250.190.46
- DNS is the system that translates human-readable domain names into IP addresses
- Think of it as the phone book of the internet
- Every time you type a URL, before any HTTP request is made, DNS resolution happens first
- Without DNS, you would have to memorize IP addresses for every website

How DNS Resolution Works – The Full Journey

- When you type google.com, your computer does not immediately know the IP
- It goes through a chain of lookups to find it
- This chain involves 4 key players – Recursive Resolver, Root Nameserver, TLD Nameserver, Authoritative Nameserver



The 4 Players in DNS Resolution

1. Recursive Resolver

- Also called a DNS Resolver or Recursive DNS Server
- This is the first stop after your machine
- Usually operated by your ISP, or you can use public ones like Google (8.8.8.8) or Cloudflare (1.1.1.1)
- Its job is to do all the legwork of finding the answer on your behalf
- It is called recursive because it keeps asking other servers until it gets the final answer
- It also caches results so it does not repeat the full lookup for every user
- Think of it as a librarian – you ask "where is google.com?" and it goes and finds the answer for you

2. Root Nameserver

- The recursive resolver first contacts a Root Nameserver
- There are only 13 sets of root nameservers in the world, labelled A through M
- They are operated by organizations like ICANN, NASA, Verisign
- The root nameserver does not know the IP of google.com
- But it knows who is responsible for .com domains
- It responds with the address of the TLD nameserver for .com
- Think of it as the top of the hierarchy – it knows who manages each extension

3. TLD Nameserver

- TLD stands for Top Level Domain – .com, .org, .in, .net, .io etc.
- The TLD nameserver for .com knows which authoritative nameserver is responsible for google.com
- It responds with the address of Google's authoritative nameserver
- It does not know the final IP either – it just knows who does

4. Authoritative Nameserver

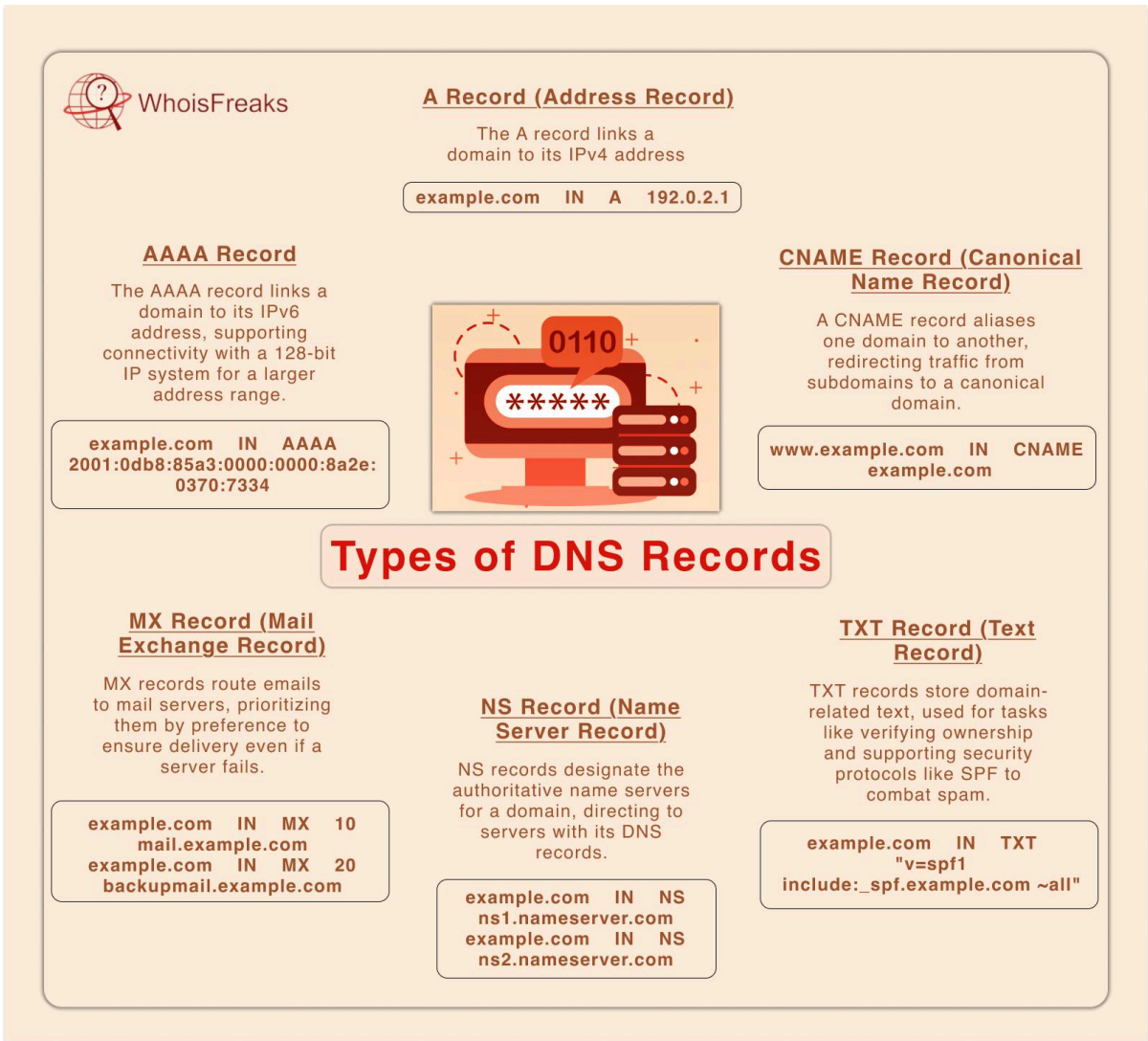
- This is the final authority for a domain
- It holds the actual DNS records for google.com
- When the recursive resolver asks "what is the IP of google.com?", this server gives the definitive answer
- It responds with the A record — the actual IP address
- This answer travels back through the recursive resolver to your machine

The Complete Resolution Flow — Step by Step

- You type google.com in your browser
- Your OS checks its local DNS cache — if found, done
- If not found, your OS asks the Recursive Resolver (your ISP or 8.8.8.8)
- Recursive Resolver checks its own cache — if found, returns it
- If not cached:
  - Recursive Resolver asks a Root Nameserver → gets back "go ask the .com TLD server"
  - Recursive Resolver asks the .com TLD Nameserver → gets back "go ask Google's authoritative nameserver"
  - Recursive Resolver asks Google's Authoritative Nameserver → gets back the IP 142.250.190.46
- Recursive Resolver caches this result and returns the IP to your machine
- Your OS caches it locally
- Browser now makes the HTTP request to 142.250.190.46
- Total time for this whole chain — typically 20 to 120 milliseconds on first lookup

DNS Record Types

- DNS records are entries stored in the authoritative nameserver
- Each record type serves a different purpose



A Record — Address Record

- Maps a domain name to an IPv4 address
- This is the most fundamental record
- Example: google.com → 142.250.190.46
- When you resolve a domain to get its IP for an HTTP request, you are fetching the A record

AAAA Record — IPv6 Address Record

- Same as A record but maps to an IPv6 address
- IPv6 addresses are 128-bit, look like 2607:f8b0:4004:c09::64
- As the internet moves toward IPv6, AAAA records become increasingly important
- Most domains today have both A and AAAA records

CNAME Record — Canonical Name Record

- Maps a domain name to another domain name, not an IP
- Example: www.example.com → example.com
- When the resolver sees a CNAME, it then resolves the target domain to get the IP
- Common use case — pointing multiple subdomains to the same underlying domain
  - blog.example.com → example.com
  - shop.example.com → example.com
- Also used with CDNs — you point your domain to the CDN's domain
  - assets.example.com → example.cdn-provider.com
- Important rule — a CNAME cannot coexist with other records on the same name
- You cannot put a CNAME on a root domain (example.com) in standard DNS — only on subdomains
  - This is why some DNS providers offer ALIAS or ANAME records as a workaround for root domains

MX Record — Mail Exchange Record

- Tells the world which mail server handles email for a domain
- When someone sends email to you@example.com, their mail server looks up the MX record of example.com to know where to deliver it
- MX records have a priority value — lower number means higher priority
- Example:
  - example.com MX 10 mail1.example.com
  - example.com MX 20 mail2.example.com
- If mail1 is down, the sender tries mail2

TXT Record — Text Record

- Stores arbitrary text data associated with a domain
- Originally for human-readable notes, now used for machine-readable verification
- Most common uses:
  - SPF (Sender Policy Framework) — declares which mail servers are allowed to send email on behalf of your domain, prevents email spoofing
  - DKIM — stores the public key used to verify email signatures
  - Domain ownership verification — Google, AWS, and others ask you to add a TXT record to prove you own the domain
- Example: v=spf1 include:_spf.google.com ~all

NS Record — Nameserver Record

- Specifies which nameservers are authoritative for a domain
- When you register a domain, you set NS records to point to your DNS provider
- Example:
  - example.com NS ns1.cloudflare.com
  - example.com NS ns2.cloudflare.com
- This tells the TLD nameserver "for questions about example.com, ask Cloudflare's nameservers"
- Changing your NS records is what you do when you switch DNS providers

PTR Record — Pointer Record

- The reverse of an A record — maps an IP address back to a domain name
- Used for reverse DNS lookup
- Example: 142.250.190.46 → google.com
- Common use case — email servers do reverse DNS lookups to verify that the sending IP matches the claimed domain, used as an anti-spam measure
- Also useful in server logs — instead of seeing raw IPs, you see domain names

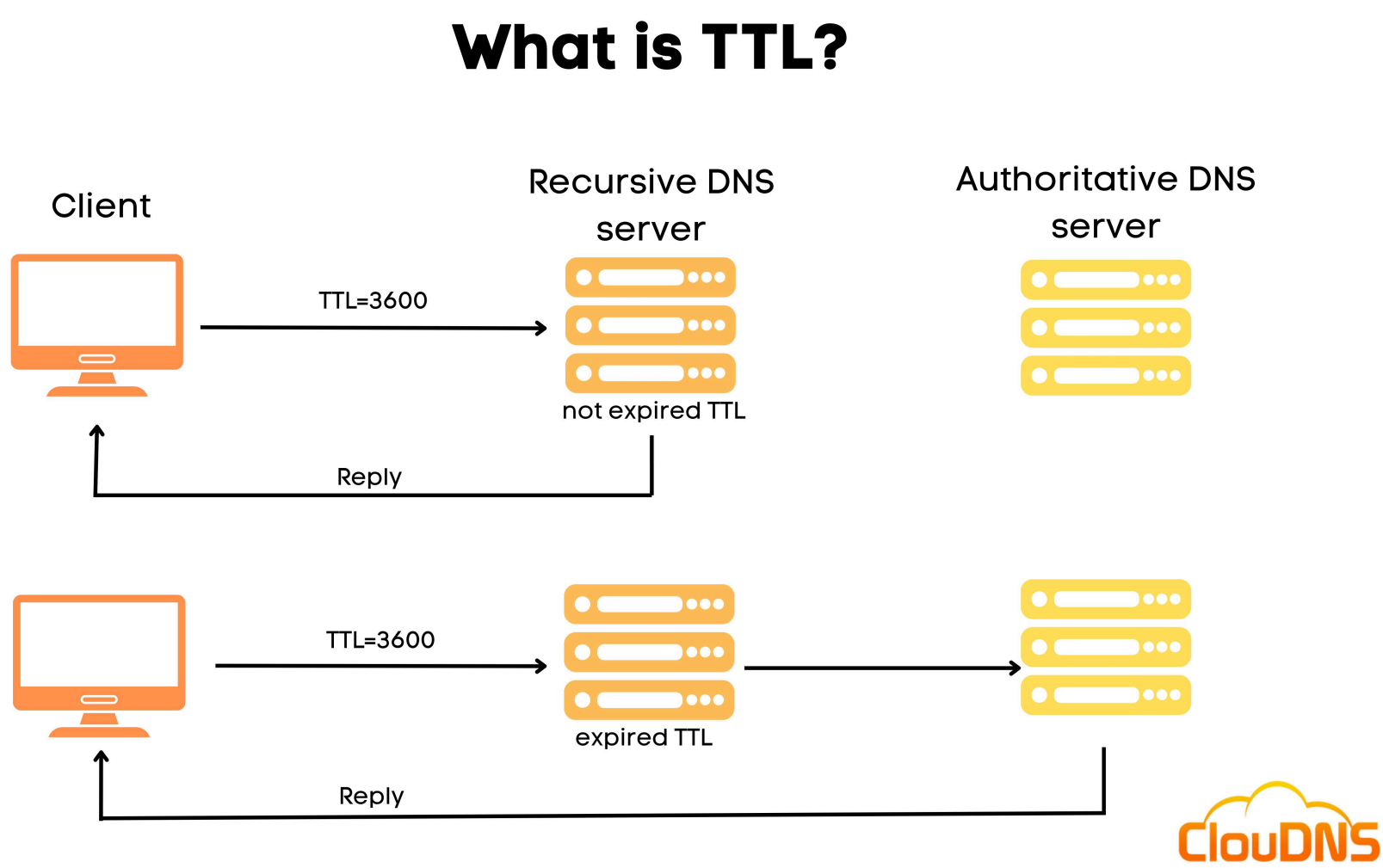
SOA Record — Start of Authority Record

- Every DNS zone has exactly one SOA record
- It contains administrative information about the zone:
  - Primary nameserver for the zone
  - Email of the zone administrator
  - Serial number — incremented every time the zone is updated, used by secondary nameservers to know when to sync
  - Refresh interval — how often secondary nameservers check for updates
  - Retry interval — how long to wait before retrying a failed refresh

    - Expire value – how long a secondary nameserver will keep serving the zone if it cannot reach the primary
    - Minimum TTL – default TTL for records in the zone
- You rarely configure this directly unless you are running your own nameserver

TTL and Caching in DNS

- TTL stands for Time To Live
- Every DNS record has a TTL value measured in seconds
- It tells resolvers and clients how long they can cache this record before checking again
- Example: A record for example.com with TTL 3600 means resolvers cache the IP for 1 hour
- After TTL expires, the next query triggers a fresh lookup



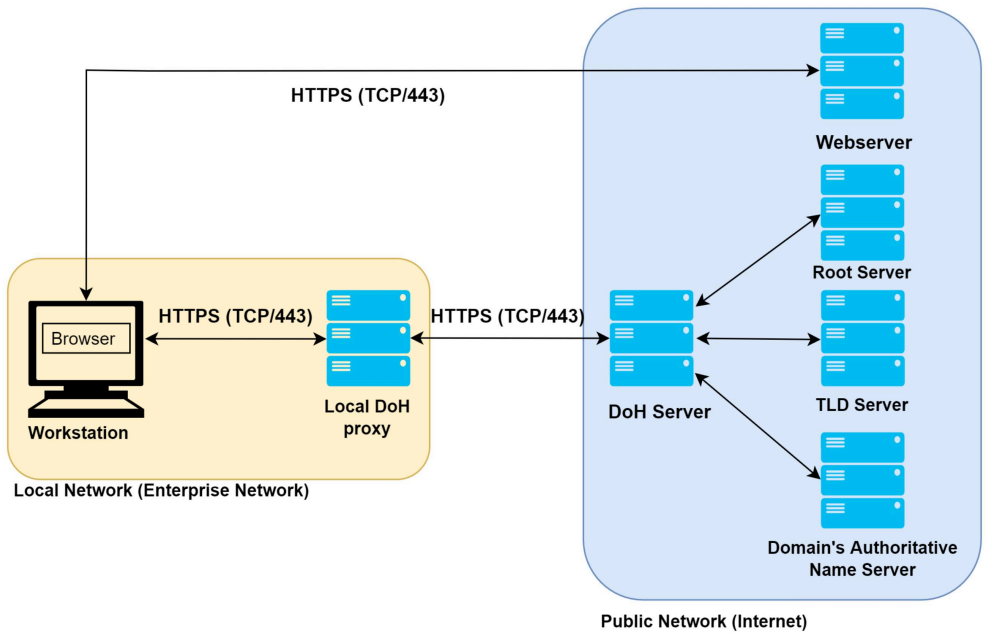
Why TTL matters in system design

- Low TTL (60–300 seconds) – changes propagate quickly, good for failover scenarios
    - If your server goes down and you point DNS to a new IP, low TTL means users switch quickly
    - Downside – more DNS queries, slightly more latency for cache misses
- High TTL (86400 seconds = 1 day) – fewer DNS queries, better performance
    - Downside – if you need to change an IP, old IPs are cached for up to a day
- Best practice before a planned migration:
    - Lower TTL to 300 seconds a day before
    - Make the DNS change
    - Once traffic has shifted, raise TTL back up

DNS Propagation

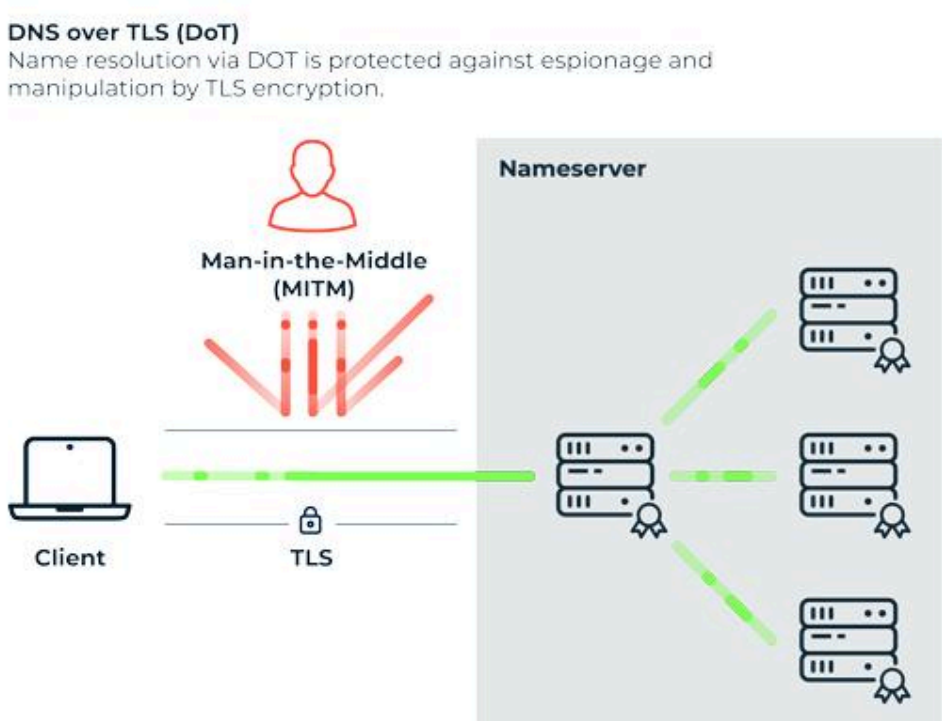
- When you update a DNS record, the change does not happen instantly everywhere
- DNS propagation is the time it takes for the updated record to spread across all resolvers worldwide
- Why it takes time:
    - Every recursive resolver that cached the old record will keep serving it until its TTL expires
    - Different resolvers have different cache states
    - Some ISPs ignore TTL and cache longer than they should
- Propagation typically takes anywhere from a few minutes (with low TTL) to 48 hours (with high TTL or misbehaving resolvers)
- You can check propagation status using tools like whatsmydns.net which shows what different resolvers around the world are returning for your domain

DNS over HTTPS (DoH)



- Traditional DNS queries are sent in plain text over UDP port 53
- Anyone on the network can see which domains you are querying — your ISP, network admin, anyone doing packet inspection
- DNS over HTTPS solves this by sending DNS queries inside HTTPS requests
- Your DNS query is encrypted and looks like regular web traffic on port 443
- Nobody in the middle can tell which domains you are resolving
- Used by browsers like Firefox and Chrome — they have their own DNS resolvers (Mozilla uses Cloudflare, Google uses its own)
- Downside — some network admins and enterprise environments need to inspect DNS traffic for security monitoring, DoH breaks that visibility

DNS over TLS (DoT)



- Same goal as DoH — encrypts DNS queries to prevent snooping
- But instead of wrapping DNS in HTTPS, it wraps DNS directly in a TLS connection
- Uses port 853 instead of port 443
- Difference from DoH:
  - DoT is easier for network admins to identify and control — it uses a dedicated port (853)
  - DoH is harder to block because it looks like regular HTTPS traffic on port 443
  - DoT is preferred in enterprise/network-level deployments
  - DoH is preferred for end-user privacy since it blends in with normal traffic
- Both achieve the same core goal — encrypted DNS queries

Split Horizon DNS

- Also called Split-Brain DNS or Split DNS
- The idea: the same domain name resolves to different IP addresses depending on where the query comes from
- Example:
  - api.example.com resolves to 10.0.0.5 (internal private IP) when queried from inside the company network
  - api.example.com resolves to 52.23.44.100 (public IP) when queried from the internet

- Why this is useful:
  - Internal traffic stays inside the private network — faster, more secure, no need to go through the public internet and back
  - You expose only one domain name but route traffic optimally based on origin
  - External users hit the public-facing load balancer or API gateway
  - Internal services, CI/CD pipelines, and developers hit the internal service directly
- How it is implemented:
  - You run two separate DNS zones for the same domain
  - Internal DNS server serves the internal zone with private IPs
  - Public DNS server serves the public zone with public IPs
  - Internal machines are configured to use the internal DNS server
  - External machines use public DNS
- Common in enterprise environments and cloud VPC setups
- AWS Route 53, Azure DNS, and GCP Cloud DNS all support split horizon DNS natively